

Компьютерная лингвистика

Конспекты по предмету

Милов Данила

ОГЛАВЛЕНИЕ

ГЛАВА 1	КОНЕЧНЫЕ ПРЕОБРАЗОВАТЕЛИ	СТРАНИЦА 2
1.1	Конечные преобразователи	2
1.2	Морфологический анализ	4
1.3	Алгоритм Портера	11
ГЛАВА 2	ОРФОГРАФИЧЕСКИЕ ОШИБКИ	СТРАНИЦА 12
2.1	Исправление орфографических ошибок	12
2.2	Формула Байеса	14
2.3	N-граммы	17
2.4	Сглаживание	20
ГЛАВА 3	ЧАСТИ РЕЧИ	СТРАНИЦА 25
3.1	Определение частей речи	25
	Способы определения частей речи — 25	
ГЛАВА 4	ГРАММАТИКИ	СТРАНИЦА 30
4.1	Контекстно-зависимые грамматики	30
4.2	Классические категориальные грамматики	32

Глава 1

Конечные преобразователи

1.1 Конечные преобразователи

Определение 1.1.1: Конечный преобразователь

Шестёрка $S = (Q, \Sigma, \Delta, q_0, \delta, F)$, где:

Q — множество состояний,

Σ — входной алфавит,

$\delta \subset Q \times \Sigma^* \times \Delta \times Q$ — конечное множество переходов(программа),

q_0 — начальное состояние,

$F \subseteq Q$ — множество закл. состояний

Определение 1.1.2: Конфигурация

Тройка (q, u, v) , где $q \in Q, u \in \Sigma^*, v \in \Delta^*$

Определение 1.1.3: Переход за один шаг($\vdash$)

$(q, u, v) \vdash_S (p, x, y)$, если существует $w \in \Sigma^*, t \in \Delta^*$:

$u = wx, y = vt$ и $q, w \rightarrow p, v \in \delta$

Определение 1.1.4: Отношение, вычисляемое S

$R(S) = \{(x, y) : (q_0, x, \varepsilon) \vdash_S^* (q_f, \varepsilon, y) \text{ для некоторого } q_f \in F\}$

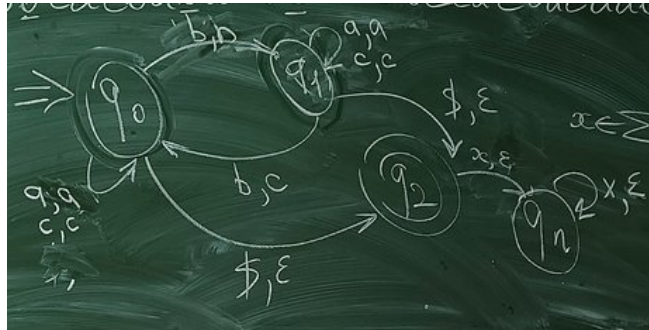
Определение 1.1.5: Детерминированный КП

Шестёрка $S = (Q, \Sigma, \Delta, \delta, q_0, F)$, где $\delta : Q \times \Sigma \rightarrow Q \times \Delta^*$

Вопрос 1: Построить КП

$\Sigma = \{a, b, c, \$\}$

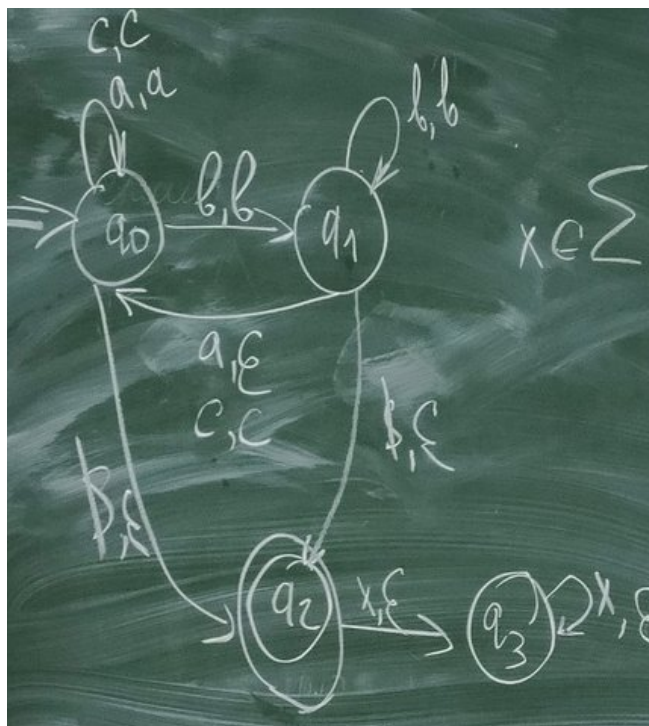
Заменить все чётные символы b на c (нумерация с 1).



Вопрос 2: Построить КП

$\Sigma = \{a, b, c, \$\}$

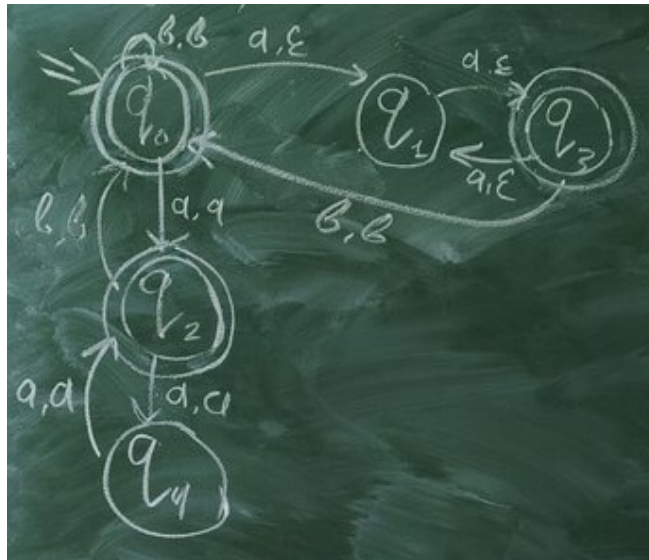
Стереть все символы a , стоящие непосредственно после символов b .



Вопрос 3: Построить КП

$\Sigma = \{a, b\}$

Удалить из x все блоки символов a чётной длины.



Note:

Домашнее задание:
№134(в,г),133,137

1.2 Морфологический анализ

Задачи морфологического анализа:

1. Дано слово, поставить его в нужную форму.
2. Дано слово, найти его начальную форму.

Предложение 1.2.1 Рассматриваемые характеристики

- N — существительное(noun)
- V — глагол(verb)
- SG — единственное число(singular)
- PL — множественное число(plural)

Пример 1.2.1

cat+N+PL $\Rightarrow$ cats
cats $\Rightarrow$ cat+N+PL

Определение 1.2.1: Морфологические правила

Определяют, как образуются формы.

Пример: английский язык, множественное число = ед. число + s.

Определение 1.2.2: Орфографические правила

Определяют, как меняется написание.

Пример — англ. язык, мн. число:

Слово заканчивается на s → вставить e.

Слово заканчивается на y → заменить на ie.

Слово является исключением → особый случай.

Определение 1.2.3: Основа

«Главная часть» слова.

Определение 1.2.4: Аффиксы

Части слова, приписываемые к основе:

1. Префиксы: пишутся спереди.
2. Суффиксы: пишутся сзади.
3. Инфиксы: пишутся внутри.
4. Циркумфиксы: спереди и сзади.

Note:

Для морфологического анализатора необходимы:

1. Словарь — список основ и аффиксов.
2. Морфотактика — правила объединения морфем.
3. Орфографические правила.

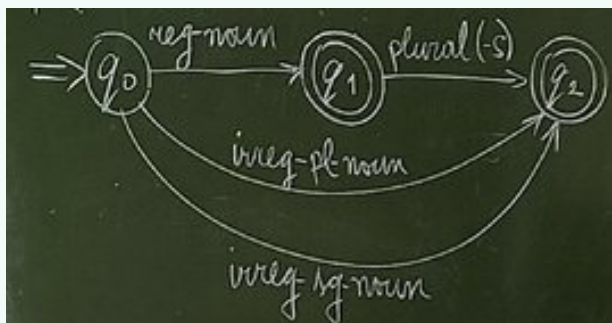
Пример 1.2.2 (Автомат, проверяющий правильность определения множественного числа)

reg-noun — правильные существительные;

irreg-sg-noun — единственное число «неправильных» существительных;

irreg-pl-noun — множественное число «неправильных» существительных;

plural(-s) — s.



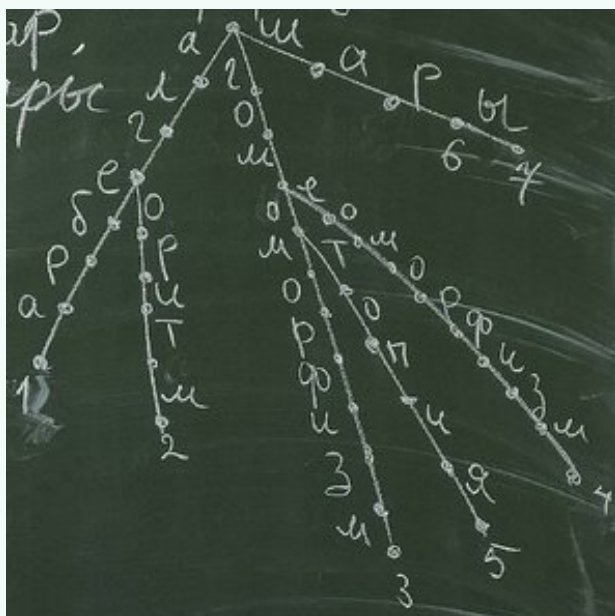
Определение 1.2.5: Префиксное дерево

Есть множество слов $S = \{S_1, \dots, S_n\}$.

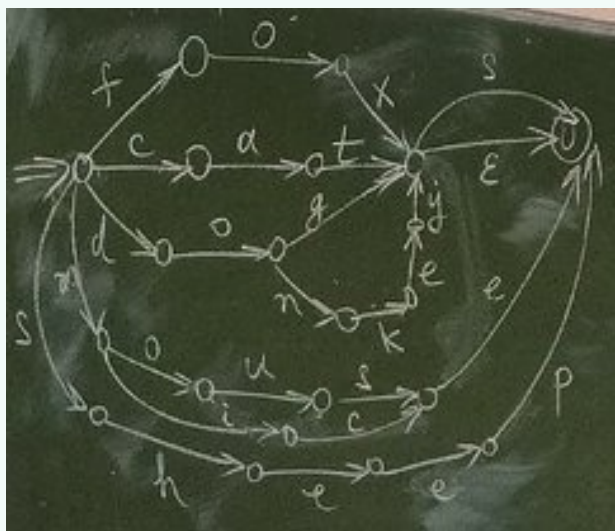
1. Рёбра дерева помечены символами.
2. Рёбра, выходящие из одной вершины, помечены разными символами.
3. На пути из корня в листья написаны слова из S .
4. Для каждого S_i существует вершина v такая, что на пути из корня в v написано S_i (v отмечены i).

Пример 1.2.3 (Префиксное дерево)

Префиксное дерево для слов: алгебра, алгоритм, гомоморфизм, гомотопия, гомеоморфизм, шар, шары.



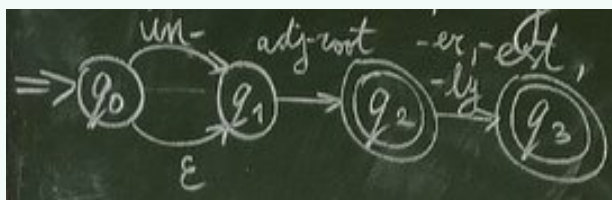
Пример 1.2.4 (Автомат, распознающий множественное число слов)



Пример 1.2.5 (Автомат для распознавания прилагательных(1))

adj-root — корень прилагательного

un-, -er, -est, -ly — аффиксы слов.



Пример 1.2.6 (Автомат для распознавания прилагательных(2))

adj-root₁ — корень прилагательного, который может употребляться с приставками

adj-root₂ — корень прилагательного, который не может употребляться с приставками

un-, -er, -est, -ly — аффиксы слов.

Это недетерминированный вариант автомата(угадываем переход по пустому слову в q_1 или q_3).



Предложение 1.2.2 Три уровня конечного преобразователя

1. Лексический: слово и его признаки. (

f	o	x	+N	+PL
---	---	---	----	-----

)
2. Промежуточный: морфемы и доп.метки (

f	o	x	^	s
---	---	---	---	---

)
3. Поверхностный: слово (

f	o	x	e	s
---	---	---	---	---

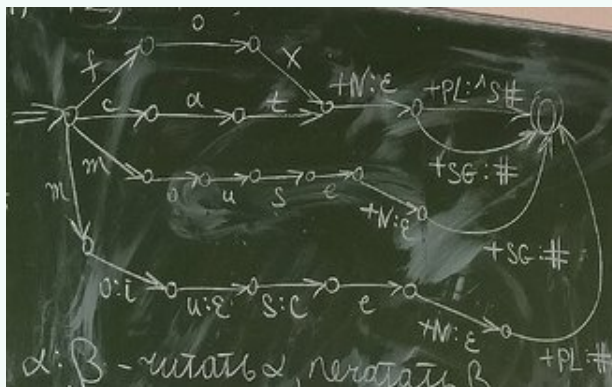
)

Пример 1.2.7 (Преобразователь слов с лексического уровня в промежуточный)

Обозначения:

$\alpha : \beta$ — читаем α , пишем β ,

α — читаем α и печатаем α .



Пример 1.2.8 (Примеры правил)

1. Замена Y. Замена -у на -ie перед -s, -i перед -ed. (*try* $\Rightarrow$ *tries*)
2. Удвоение согласной перед -ing/-ed. (*bed* $\Rightarrow$ *begging*)
3. Удаление -e перед -ing,-ed. (*make* $\Rightarrow$ *making*)
4. Вставка -e после -s,-z,-x,-ch,-sh перед -s. (*watch* $\Rightarrow$ *watches*)

Предложение 1.2.3 Запись правил

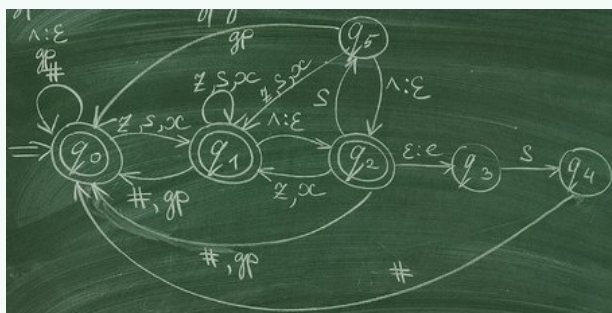
$a \Rightarrow b/c_d$ — Заменить a на b между c и d.

a,b,c,d — строки.

$cad \Rightarrow cbd$

Пример 1.2.9 (КП для распознавания особых случаев образования множественного числа)

Символы: $s, z, x, ^\wedge, \#$, др — любой другой символ.



Определение 1.2.6: Обратное отношение

$S = (Q, \Sigma, \Delta, \delta, q_0, F)$, $R(S) \subseteq \Sigma^* \times \Delta$ — конечный преобразователь.

$S' = (Q, \Delta, \Sigma, \delta', q_0, F)$ — конечный преобразователь, для которого множества входных и выходных данных поменяны местами.

А программа имеет вид:

$$q, x \rightarrow p, y \in \delta \Leftrightarrow q, y \rightarrow p, x \in \delta'$$

Тогда:

$$R(S') = (R(S))^{-1}$$

$$(x, y) \in R(S') \Leftrightarrow (y, x) \in R(S)$$

Предложение 1.2.4 Морфологический анализ слов

Конечный преобразователь из примера выше может быть использован не только для того, чтобы проверить правильность образования формы множественного числа, но и для того, чтобы на его основе построить автомат S' , обратный данному, восстанавливающий второй уровень слова по третьему.

Пример: выделим морфемы в словах cats и foxes. Для этого построим автомат, вычисляющий отношение, обратное $R(S)$.

c	a	t	s	#	
q_0	q_0	q_0	q_0	q_1	
c	a	t	$^\wedge$	s	#

f	o	x	e		s	#	
q_0	q_0	q_0	q_1	q_2	q_3	q_4	q_0
f	o	x	$^\wedge$	ϵ	s	#	

1.3 Алгоритм Портера

Note:

Алгоритм Портера в презентации Бориса Николаевича

Вопрос 4: Применить алгоритм

Слово: **compiling**

1. —
2. 2.a) compil
2.б) —
3. —
4. —
5. —
6. —
7. —

Вопрос 5: Применить алгоритм

Слово: **comlipation**

1. —
2. —
3. —
4. compile
5. —
6. compil
7. —

Note:

Д/з: прогнать алгоритм Портера для слов:
formalization, axiomatization, enumeration, fuzzieness, finger, singer

Глава 2

Орфографические ошибки

2.1 Исправление орфографических ошибок

Предложение 2.1.1 Виды орфографических ошибок

1. **Не слова:** ошибка приводит к слову, которого нет.
2. **Не то слово:** конечный автомат $\rightarrow$ конечный автомат.

Предложение 2.1.2 Исправление ошибок

1. **Изолированное исправление** исправляем одно слово, контекст игнорируется.
2. **С учётом контекста.**

Определение 2.1.1: Выравнивание строк

$S[1 \dots n], T[1 \dots m] \in \Sigma$

$- \notin \Sigma$

σ — стоимость однобуквенного выравнивания.

$\sigma(a, a) \geq 0$

$\sigma(-, a) \leq 0$

$\sigma(a, -) \leq 0$

$\sigma(-, -) \leq 0$

Выравнивание строк: $S'[1 \dots l], T'[1 \dots l]$ такие, что S, T — проекции S', T' на Σ

Определение 2.1.2: Стоимость выравнивания S', T'

$$\sigma(S', T') = \sum_{i=1} \sigma(S'[i], T'[i])$$

Определение 2.1.3: Оптимальное выравнивание

Выравнивание наибольшей стоимости.

Определение 2.1.4: Редакционное расстояние

Наименьшее число операций вставки символа, удаления символа и замены символа для преобразования S в T .

Определение 2.1.5: Алгоритм Вагнера-Фишера

$V(i, j)$ — наибольшая стоимость выравнивания $S[1 \dots i]$ и $T[1 \dots j]$

$$V(0, 0) = 0$$

$$V(i + 1, 0) = V(i, 0) + \sigma(S[i + 1], -)$$

$$V(0, j + 1) = V(0, j) + \sigma(-, T[j + 1])$$

$$V(i + 1, j + 1) = \max\{V(i, j) + \sigma(S[i + 1], T[j + 1]),$$

$$V(i + 1, j) + \sigma(-, T[j + 1]),$$

$$V(i, j + 1) + \sigma(S[i + 1], -), \}$$

Время: $O(mn)$

Память: $O(mn)$, если нужно само выравнивание

$O(\min\{m, n\})$, если нужна только стоимость

Предложение 2.1.3 Версия алгоритма с хорошей памятью

$W(i, j)$ — наибольшая стоимость выравнивания $S[n - i + 1 \dots n]$ и $T[m - j + 1 \dots m]$

Note:

Задача. Написать рекурсивные соотношения для W

$$W(m, n) = V(m, n)$$

Время W : $O(mn)$

$$m \leq n$$

1. Если $m = 1$, то найти такое i , что $\sigma(S[i], T[1])$ максимально. То есть выбрать один из вариантов:

•

2. Пусть $m > 1$.

3. Пусть $T_1 = T[1 \dots \frac{m}{2}]$, $T_2 = T[\frac{m}{2} + 1 \dots m]$.
4. Вычислять последние строки таблицы V для S и T_1 , таблицы W для S и T_2
5. Найти такое i , что $V(i, \frac{m}{2}) + W(n - i, \frac{m}{2})$ максимально.
6. Рекурсивно найти оптимальное выравнивание $S[1 \dots i]$ и T_1 , $S[i + 1 \dots n]$ и T_2 .

Утверждение 2.1.1

$$T(n, m) \leq 2mn$$

Доказательство: $T(n, m) = mn + T(i, \frac{m}{2}) + T(n - i, \frac{m}{2})$
 S и T_1 — $\frac{m}{2}n$, S и T_2 — $\frac{m}{2}n$, всего — mn



Утверждение 2.1.2

Когда i найдено, V и W больше не нужны. Вычисляем V и W в глобальном массиве размера $O(m) = O(\min\{m, n\})$.

Число рекурсивных вызовов $\leq \log_2 m$.

Память под стек: $O(\log_2 m)$

Общая память: $O(m) = O(\min\{m, n\})$

2.2 Формула Байеса

Определение 2.2.1: Условная вероятность

$$P(A|B) = \frac{P(AB)}{P(B)}$$

Определение 2.2.2: Полная группа событий

$$\Omega = H_1 \cup H_2 \cup \dots \cup H_m, H_i \cap H_j = \emptyset$$

Определение 2.2.3: Независимые события

$$P(A|B) = P(A) \text{ или } P(AB) = P(A)P(B)$$

Определение 2.2.4: Формула полной вероятности

$$P(A) = \sum_{i=1} P(A \setminus H_i) P(H_i)$$

Определение 2.2.5: Формула Байеса

$$P(A \setminus B) = \frac{P(B \setminus A) P(A)}{P(B)}$$
$$P(H_i \setminus A) = \frac{P(A \setminus H_i) P(H_i)}{P(A)} = \frac{P(A \setminus H_i) P(H_i)}{\sum_{i=1}^n P(A \setminus H_i) P(H_i)}$$

Предложение 2.2.1 Применение формулы Байеса

c — правильное написание слова. t — слово с ошибкой. $P(c \setminus t)$ — вероятность того, что вместо c написать t

$$P(c \setminus t) = \frac{P(t \setminus c) P(c)}{P(t)}$$

$P(c)$ — вероятность употребления слова c

$P(t)$ — вероятность написать t

$P(t \setminus c)$ — вероятность допустить ошибку, которая превращает c в t

Определение 2.2.6: Корпус

Набор текстов (возможно размеченных)

Note:

$P(c)$ приблизительно равно частоте слова в корпусе

$P(t \setminus c)$

Пример 2.2.1 (Типичные ошибки)

1. Вставка символа: количество $\Rightarrow$ колличество
2. Удаление: солнце $\Rightarrow$ сонце
3. Замена: единица $\Rightarrow$ еденица
4. Перестановка соседних символов: конечный $\Rightarrow$ конченый

Предложение 2.2.2 Определим некоторые величины

- $del[x, y]$ — число раз, когда вместо xy было написано x

- $ins[x, y]$ — число раз, когда вместо x было написано xy
- $sub[x, y]$ — число раз, когда вместо x было написано y
- $trans[x, y]$ — число раз, когда вместо xy было написано yx
- $count[\alpha]$ — число раз, когда написано α

$$P(t \setminus c) = \begin{cases} \frac{del[c_{p-1}, c_p]}{count[c_{p-1}c_p]} & \text{для удаления,} \\ \frac{ins[c_{p-1}, t_p]}{count[c_{p-1}]} & \text{для вставки,} \\ \frac{sub[c_p, t_p]}{count[c_p]} & \text{для замены,} \\ \frac{trans[c_p, c_{p+1}]}{count[c_p c_{p+1}]} & \text{для перестановки} \end{cases}$$

Определение 2.2.7: Матрица ошибок

Матрица значений $del, ins, sub, trans, count$.

Note:

Возвращаясь к формуле Байеса для поиска вероятности ошибок, в формуле

$$P(c \setminus t) = \frac{P(t \setminus c)P(c)}{P(t)}$$

Мы научились вычислять $P(c), P(t \setminus c)$.

А вычислять $P(t)$ не нужно, так как

$$\hat{c} = \operatorname{argmax} P(c \setminus t) = \operatorname{argmax} \frac{P(t \setminus c)P(c)}{P(t)} = \operatorname{argmax} P(t \setminus c)P(c)$$

Вопрос 6: Найти вероятности исправлений

Дано слово **стор**

Есть альтернативы:

1. стар — 0.2%
2. стог — 0.1%
3. стон — 0.3%
4. сток — 0.5%
5. стоп — 0.1%
6. тор — 1%
7. сто — 0.2%
8. сор — 0.3%

9. створ — 0.3%

Подсчёт вероятностей ошибок для каждого из слов:

1. $P(t \setminus c) = 1\%, P(c) = 0.2\%$

$$P(\text{стор} \setminus \text{стар})P(\text{стар}) = 2 \cdot 10^{-5}$$

2. $P(\text{стор} \setminus \text{стог})P(\text{стог}) = 10^{-5}$

3. $3 \cdot 10^{-5}$

4. $5 \cdot 10^{-5}$

5. 10^{-5}

6. $3 \cdot 10^{-4}$ — слово с наибольшей вероятностью.

7. $6 \cdot 10^{-5}$

8. $9 \cdot 10^{-5}$

9. $6 \cdot 10^{-5}$

Получается, что наиболее вероятное исправление — тор

2.3 N-граммы

Определение 2.3.1: N-грамма

N-грамма — последовательность из N слов.

N=1 — униграмма

N=2 — биграмма

N=3 — триграмма

Определение 2.3.2: Скалярное произведение

$$x = (x_1, \dots, x_n)$$

$$y = (y_1, \dots, y_n)$$

$$\text{Скалярное произведение: } \sum_{i=1}^n x_i y_i = (x, y)$$

Пример 2.3.1

T_1, T_2 — два текста

n — число слов в текстах

x_i — число вхождений i -го слова в T_1

y_i — то же, что x_i , но для T_2

(x, y) — косинусное расстояние между T_1 и T_2 .

Косинусное потому что $\cos(\phi) = \frac{(x, y)}{|x| \cdot |y|}$

Идея состоит в том, что если T_1 и T_2 "близки" то (x, y) — большое. А если T_1 и T_2 не связаны тематически, то (x, y) близко к 0.

Возникают вопросы, связанные с тем, что слова имеют много форм:

1. Можно убрать из T_1, T_2 артикли, предлоги, союзы и т.п.
2. Нужно ли различать формы одного слова.

Предложение 2.3.1 Подсчёт вероятностей последовательностей слов

$$P(AB) = P(A|B)P(B)$$

$$P(ABC) = P(A|BC)P(B|C)P(C)$$

$(w_1, \dots, w_n)$ — последовательность слов(текст)

$P(w_1, \dots, w_n)$ — вероятность $(w_1, \dots, w_n)$

$$P(w_1, \dots, w_n) = P(w_1)P(w_2|w_1)P(w_3|w_2w_1) \dots P(w_n|w_1 \dots w_{n-1})$$

В данной ситуации помогут N-граммы.

Предположим, что слово w_i зависит только от N-1 предыдущего слова.

$$P(w_i|w_1, \dots, w_{i-1}) = P(w_i|w_{i-N+1} \dots w_{i-1})$$

Пример 2.3.2 (Частные случаи N-грамм)

1. N=1.

$$P(w_i|w_1 \dots w_{i-1}) = P(w_i)$$

2. N=2.

$$P(w_i|w_1 \dots w_{i-1}) = P(w_i|w_{i-1})$$

3. N=3.

$$P(w_i|w_1 \dots w_{i-1}) = P(w_i|w_{i-2}w_{i-1})$$

Note:

Если в языке V слов, то число N-грамм составляет V^N . Тогда возникает проблема:

- Если N "маленькое" то не учитываются "длинные" контексты.
- Если N "большое" то V^N больше размера корпуса.

При "больших" N получается переобучение.

$$P(w_1, \dots, w_n) = \prod_{i=1}^n P(w_i|w_{i-N+1} \dots w_{i-1})$$

$w_{N-2} = w_{N-1} = \dots = w_0 = < s >$ — спец слово

$$\text{При } N = 2. P(w_1, \dots, w_n) = P(w_1)p(w_2|w_1)P(w_3|w_2) \dots P(w_n|w_{n-1})$$

Пример 2.3.3 (Переобучение)

Пусть есть некоторые результаты эксперимента (точки на графике). По этим результатам мы хотим примерно оценить исходную функцию (в примере $y = ax$). Однако через любое конечное количество точек может проходить многочлен, как изображено на рисунке:



Предложение 2.3.2

$c(xy)$ — число пар xy в корпусе

$c(x)$ — число слов x в корпусе

$$P(w_i|w_{i-1}) = \frac{c(w_{i-1}w_i)}{\sum_w c(w_{i-1}w_i)} = \frac{c(w_{i-1}w_i)}{c(w_{i-1})}$$

Пример 2.3.4

Ворон к ворону летит, ворон ворону кричит.

1. Различать ли строчные и заглавные буквы?
2. Учитывать ли знаки препинания?
3. Различать ли формы слова?

2.4 Сглаживание

Определение 2.4.1: Сглаживание

При большом N часто $c(w_{i-1}w_i) = 0, P(w_i|w_{i-1}) = 0$.

0 могут быть в знаменателе в формулах, указанных выше.

Сглаживание — изменение вероятностей так, чтобы:

1. не осталось нулей;
2. вероятности изменились "не очень сильно".

Note:

Пусть есть V наблюдаемых объектов (N -граммы).

Проведено K наблюдений: $\{a_1, \dots, a_V\}$.

c_i — число наблюдений a_i

p_i — вероятность a_i

$$p_i = \frac{c_i}{K}$$

Предложение 2.4.1 Аддитивное сглаживание (Сглаживание Лапласа)

Считаем, что каждый объект наблюдается на один раз чаще, чем на самом деле.

p_i^* — сглаживание вероятностей.

$$p_i^* = \frac{c_i + 1}{K + V}$$

$K + V$ — число наблюдений с учётом добавленных

$$\sum_{i=1}^V p_i^* = \sum_{i=1}^V \frac{c_i + 1}{K + V} = \frac{1}{K + V} \sum_{i=1}^V c_i + \sum_{i=1}^V \frac{1}{K + V} =$$

$$= \frac{K}{K + V} + \frac{V}{K + V} = 1$$

получается, что $p_i^* \geq 0$

c_i^* — сглаживание количества. $c_i^* = p_i^* K = (c_i + 1) \frac{K}{K + V}$

$$\sum_{i=1}^V c_i^* = \sum_{i=1}^V p_i^* K = K$$

Вопрос 7: Применить сглаживание Лапласа

Дан корпус:

ворон к ворону летит,

ворон ворону кричит,

ворон ворону в ответ

Построим исходную таблицу N-грамм:

	ворон	к	ворону	летит	кричит	в	ответ
ворон		1;0.1;	2;0.2;				
к			1;0.1;				
ворону				1;0.1	1;0.1;	1;0.1;	
летит							
кричит							
в							1;0.1;
ответ							

Применим сглаживание:

Подсчитаем вероятности:

В пустых ячейках вероятность равна $\frac{1}{59} \approx 0.017$

В ячейках с вероятностью 0.1 вероятность равна $\frac{2}{59} \approx 0.034$

В ячейках с вероятностью 0.2 вероятность равна $\frac{3}{59} \approx 0.051$

Подсчитаем количество:

В пустых ячейках количество равно $\frac{10}{59} \approx 0.169$

В ячейках с вероятностью 0.1 вероятность равна $\frac{20}{59} \approx 0.339$

В ячейках с вероятностью 0.2 вероятность равна $\frac{30}{59} \approx 0.508$

	ворон	к	ворону	летит
ворон		0.339;0.034;	0.508;0.051;	
к			0.339;0.034;	
ворону				0.339;0.034;
летит	0.339;0.034;			
кричит	0.339;0.034;			
в				
ответ				

	кричит	в	ответ
ворон			
к			
ворону	0.339;0.034;	0.339;0.034;	
летит			
кричит			
в			0.339;0.034;
ответ			

Предложение 2.4.2 Сглаживание с учётом контекста

Сглаживаем построчно, фиксируя первое слово в каждой строке.

Первая строка:

$$V = 7, K = 3$$

$$c_{1,2} = 1, c_{1,3} = 2$$

$$p_{1,2} = \frac{1}{3}, p_{1,3} = \frac{2}{3}$$

$$p_{1,1}^* = \frac{0+1}{7+3} = \frac{1}{10}$$

$$p_{1,2}^* = \frac{1+1}{7+3} = \frac{2}{10} = \frac{1}{5}$$

$$p_{1,3}^* = \frac{2+1}{7+3} = \frac{3}{10}$$

$$c_{1,1}^* = \frac{3}{10}$$

$$c_{1,2}^* = \frac{3}{5}$$

$$c_{1,3}^* = \frac{9}{10}$$

Вторая строка: $V = 7, K = 1$

$$p_{2,1}^* = \frac{0+1}{7+1} = \frac{1}{8}$$

$$c_{2,1}^* = \frac{1}{8}$$

$$p_{2,3}^* = \frac{1+1}{7+1} = \frac{1}{4}$$

$$c_{2,3}^* = \frac{1}{4}$$

Предложение 2.4.3 Сглаживание Уиттена-Белла

V, K, c_i — означают то же, что и в предыдущем алгоритме.

Суть алгоритма:

Оценить вероятность того, что на $(K+1)$ -м шаге появится a_i .

Вероятность нового $a_i \approx$ вероятность появления события впервые.

Введём величину:

$Z = \sum_{c_i=0} 1$ — количество событий, которые никогда не наблюдались. Тогда дадим всем им такую вероятность:

$\sum_{c_i=0} p_i^* = \frac{T}{K+T}$, где T — количество событий, которые произошли хотя бы один раз.

$\frac{T}{K+T}$ — вероятность увидеть событие впервые.

Тогда вероятности определяются следующим образом:

$$p_i^* = \frac{T}{Z(K+T)}, \text{ если } c_i = 0$$

$$p_i^* = \frac{c_i}{K+T}, \text{ если } c_i > 0$$

Количества вычисляются следующим образом:

$$c_i^* = p_i^* K$$

$$c_i^* = \frac{T}{Z} \cdot \frac{K}{K+T}, \text{ если } c_i = 0$$

$$c_i^* = c_i \cdot \frac{K}{K+T}, \text{ если } c_i > 0$$

Вопрос 8: Сглаживание Уиттена-Белла

Применим к примеру с вороном сглаживание Уиттена-Белла. $V = 7^2 = 49, K = 10, T = 9$

$$Z = V - T = 49 - 9 = 40$$

$$p_{1,1}^* = \frac{T}{Z(K+T)} = \frac{9}{40(10+9)} \approx 0.012$$

$$p_{1,2}^* = \frac{c_i}{K+T} = \frac{1}{19} \approx 0.053$$

$$p_{1,2}^* = \frac{2}{19} \approx 0.105$$

$$c_{1,1}^* = 0.12$$

$$c_{1,2}^* = 0.53$$

$$c_{1,3}^* = 1.05$$

Предложение 2.4.4 Метод Гуда-Тьюринга

K_c — количество событий, которые произошли ровно c раз.

K_0 — число событий, которые не наблюдались.

Последнее наблюдение произошло $c + 1$ раз.

Вероятность, что последний элемент наблюдался $c + 1$ раз $\approx \frac{(c+1)K_{c+1}}{K_c}$

Вероятность того, что c превратится в $c + 1 \approx \frac{(c + 1)K_{c+1}}{K_c}$

$$c = 0, p_0^* = \frac{K_1}{K}$$

Имеем K_r количество элементов частоты r

$$p_{r+1}^* = \frac{(r + 1)K_{r+1}}{KK_r}$$

$$\text{Тогда } c_{r+1}^* = p_{r+1}^*, K = (r + 1) \frac{K_{r+1}}{K_r}$$

Пример 2.4.1 (Метод Гуда-Тьюринга для примера с вороном)

Глава 3

Части речи

3.1 Определение частей речи

3.1.1 Определение, основанное на правилах

Пример 3.1.1 (that)

I consider that odd. — "это"

It isn't that odd. — "настолько"

если (+1 A/ADV/QUANT)(+2 SENT-LIM)(NOT -1 SVOC/A) то{
удалить все отличные от ADV
иначе
удалить метку ADV
}

Note:

+1 — слово справа
+2 — второе слово справа
A — прилагательное(adjective)
ADV — наречие(adverb)
QUANT — квантификатор
SENT-LIM — граница предложения
-1 — слово слева
SVOC — глагол типа consider

3.1.2 Скрытые марковские модели

Определение 3.1.1: Цепь маркова

$Q = \{q_0, \dots, q_n\}$ — состояния

$A = (a_{ij})$ — вероятности переходов

a_{ij} — вероятность перехода из q_i в q_j

$$\sum_{j=0}^n a_{ij} = 1 \text{ для всех } i$$

Марковское свойство: вероятность перехода определяется только текущим состоянием и не зависит от предыдущих состояний.

$\pi(0 \leq i \leq n)$ — начальное распределение.

π_i — вероятность начать в состоянии q_i , $\sum_{i=0}^n \pi_i = 1$

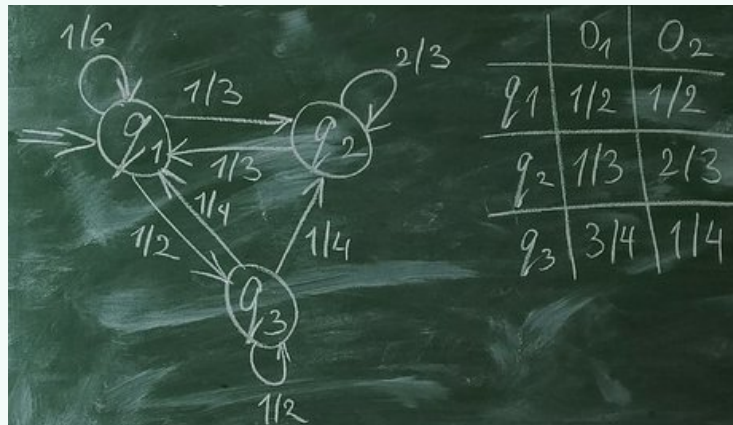
Определение 3.1.2: Скрытая марковская модель

Q, A, π — как в цепи Маркова

$O = \{o_1, \dots, o_m\}$

$B = b_i(o_t)$ — вероятность наблюдения o_t в состоянии q_i .

Пример 3.1.2 (Пример скрытой марковской модели)



Note:

Называется скрытой, потому что состояния сами по себе не видны, видны только наблюдения.

Пример 3.1.3 (Пример задачи для СММ)

Дана последовательность наблюдений $o_1, \dots, o_T$. Найти наиболее вероятный путь в СММ.

3.1.3 Алгоритм Витерби

Предложение 3.1.1 Алгоритм

$Q = \{q_1, \dots, q_n\}$ — состояние

$o_1, \dots, o_T$ — последовательность наблюдений

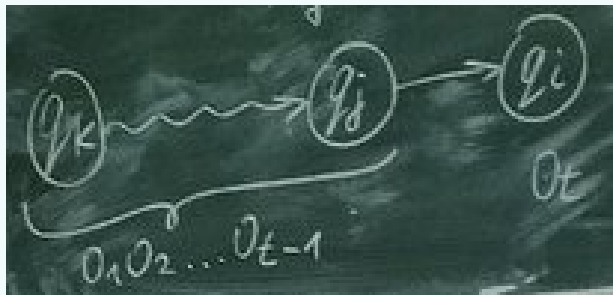
π_i — вероятность начать в q_i

1. $V(i, j)$ — максимальная вероятность перейти в q_i и выдать при этом

$o_1, \dots, o_j$

2. $V(i, 1) = \pi_i b_i(o_1)$

$$V(i, t) = \max_{1 \leq j \leq n} \{V(j, t-1) \cdot a_{ij} \cdot b_i(o_t)\} = \max_{1 \leq j \leq n} \{V(j, t-1) \cdot a_{ij}\} \cdot b_i(o_t)$$



Algorithm 1 Витерби

Ввод: Скрытая марковская модель

Вывод: Вероятности

```
1: для  $i=1$  до  $n$  {
2:    $V(i, 1) = \pi_i \cdot b_i(o_i)$ ;
3:    $p(i, 1) = null$ ;
4: }
5: для  $t = 2$  до  $T$  {
6:   для  $i = 1$  до  $n$  {
7:     для  $j = 1$  до  $n$  {
8:        $c = V(j, t-1) \cdot a_{ij} \cdot b_i(o_t)$ ;
9:       если  $V(i, t) = 0$  или  $c > V(i, t)$  то{
10:         $V(i, t) = c$ ;
11:         $p(i, t) = j$ ;
12:      }
13:    }
14:  }
15: }
```

Предложение 3.1.2 Применение алгоритма Витерби для распознавания частей речи

Q — части речи

O — слова

a_{ij} — вероятность части речи j после части речи i

$b_i(o_t)$ — вероятность слова o_t , если известно, что часть речи — i

$b_{\text{гл}}(\text{печь})$ — вероятность слова "печь если часть речи – глагол

$b_{\text{сущ}}(\text{печь})$ — вероятность слова "печь если часть речи – существительное

$a_{ij}, b_i(o_t), \pi_i$ вычисляются из корпуса. π_i — вероятность того, что первое слово имеет часть речи i .

$o_1, \dots, o_T$ — предложение. Вычисляем наиболее вероятные части речи $q_1, \dots, q_T$

Вопрос 9: Применить алгоритм Витерби

Пусть СММ из примера 3.1.2 отработала и получена последовательность наблюдений: $o_1, \dots, o_4 = abba$. Применить к ним алгоритм Витерби.

$q \backslash t$	1	2	3	4
q_1	$\frac{1}{2}, -$	$\frac{1}{24}, q_1$	$\frac{1}{54}, q_2$	
q_2	$0, -$	$\frac{1}{9}, q_1$	$\frac{4}{81}, q_2$	
q_3	$0, -$	$\frac{1}{16}, q_1$	$\frac{1}{128}, q_3$	

$$V(i, t) = \max\{V(j, t-1)a_{ij}b(o_t)\}$$

$$V(1, 2)$$

$$V(1, 1)a_{1,1}b_1(o_2) = \frac{1}{2} \cdot \frac{1}{6} \cdot \frac{1}{2} = \frac{1}{24}$$

$$V(2, 1)a_{2,1}b_1(o_2) = 0$$

$$V(3, 1)a_{3,1}b_1(o_2) = 0$$

$$V(2, 2)$$

$$V(1, 1)a_{1,2}b_2(o_2) = \frac{1}{9}$$

$$V(3, 2)$$

$$V(1, 1)a_{1,3}b_3(o_2) = \frac{1}{16}$$

$$V(1, 3)$$

$$V(1, 2)a_{1,1}b_1(o_3) = \frac{1}{288}$$

$$V(2, 2)a_{2,1}b_1(o_3) = \frac{1}{54}$$

$$V(3, 2)a_{3,1}b_1(o_3) = \frac{1}{128}$$

$$V(2, 3)$$

$$V(1, 2)a_{1,2}b_2(o_3) = \frac{1}{108}$$

$$V(2, 2)a_{2,2}b_2(o_3) = \frac{4}{81}$$

$$V(3, 2)a_{3,2}b_2(o_3) = \frac{1}{96}$$

$$V(3, 3)$$

$$V(1, 2)a_{1,3}b_3(o_3) = \frac{1}{192}$$

$$V(2, 2)a_{2,3}b_3(o_3) = 0$$

$$V(3, 2)a_{3,3}b_3(o_3) = \frac{1}{128}$$

Note:

Д/з: заполнить 4-й столбец таблицы

Глава 4

Грамматики

4.1 Контекстно-зависимые грамматики

Определение 4.1.1: КЗ-грамматики

$$G = (N, \Sigma, P, S)$$

Правила имеют вид: $\xi A \eta \rightarrow \xi \alpha \eta$, где $A \in N$; $\xi, \eta \in (\Sigma \cup N)^*$; $\alpha \in (\Sigma \cup N)^+$

Определение 4.1.2: Неукорчивающая грамматика

$$\alpha \rightarrow \beta \in P \Rightarrow |\alpha| \leq |\beta|$$

Пример 4.1.1 (Пример КЗ-грамматики)

Язык $L_1 = \{a^n b^n c^n : n > 0\}$

Грамматика, порождающая этот язык:

$$S \rightarrow aSBC$$

$$S \rightarrow aBC$$

$$CB \rightarrow BC$$

$$aB \rightarrow ab$$

$$bB \rightarrow bb$$

$$bC \rightarrow bc$$

$$cC \rightarrow cc$$

Она не совсем КЗ. Чтобы она стала совсем КЗ, надо, чтобы правило $CB \rightarrow BC$ было заменено последовательностью правил:

$$CB \rightarrow C'B$$

$$C'B \rightarrow C'C$$

$$C'C \rightarrow BC$$

Пример 4.1.2 (Ещё пример грамматики)

$$L_2 = \{a^{n^2} : n > 0\}$$

$$S \rightarrow AB B D F$$

$$B D \rightarrow D C B$$

$$B C \rightarrow C B$$

$$A D \rightarrow A A E$$

$$E C \rightarrow A E$$

$$E B \rightarrow B E$$

$$E F \rightarrow B B D F$$

$$D F \rightarrow a$$

$$B \rightarrow a$$

$$A \rightarrow a$$

$$S \rightarrow a$$

Утверждение 4.1.1

$$S \Rightarrow A^{n^2} B^{2n} D F$$

Доказательство: $(n + 1)^2 = n^2 + 2n + 1$

$$S \Rightarrow^* A^{n^2} B^{2n} D F$$



Вопрос 10: Исправить грамматику из примера 3.2.2

Заменить правило: $D F \rightarrow a$ на $E F \rightarrow B B a$ и добавить правило $S \rightarrow a$.

Вопрос 11: Построить грамматики

$$L_3 = \{w c w : w \in \{a, b\}^*\}$$

$$L_4 = \{w w : w \in \{a, b\}^+\}$$

$$L_5 = \{a^{2^n} : n > 0\}$$

$$L_6 = \{a^n : n \text{ — составное число}\}$$

4.2 Классические категориальные грамматики

Определение 4.2.1: Элементарные категории

$\mathbb{C}$ — элементарные категории (примитивные типы). Например, часть речи в предложении.

Определение 4.2.2: Категория

1. Если $\Phi \in \mathbb{C}$, то Φ — категория.
 2. Если Φ, Ψ — категории, то $[\Phi/\Psi]$ и $[\Psi\backslash\Phi]$ — категории.
- $Cat(\mathbb{C})$ — множество всех категорий над $\mathbb{C}$.

Предложение 4.2.1 Правила сокращения

$\Gamma_1, \Gamma_2 \in Cat(\mathbb{C})^*$

1. $\Gamma_1[\Phi/\Psi]\Psi\Gamma_2 \vdash \Gamma_1\Phi\Gamma_2$
2. $\Gamma_1\Psi[\Psi\backslash\Phi]\Gamma_2 \vdash \Gamma_1\Phi\Gamma_2$

Определение 4.2.3: Классическая категориальная грамматика

Четвёрка $G = (\Sigma, \mathbb{C}, S, \lambda)$, где:

Σ — алфавит терминалов;

$\mathbb{C}$ — множество элементарных категорий;

$S \in \mathbb{C}$ — главная категория;

$\lambda : \Sigma \rightarrow P_{fin}(Cat(\mathbb{C}))$ — словарь

Note:

$$w = a_1 a_2 \dots a_n$$

$$\lambda(w) = \{\Phi_1, \Phi_2, \dots, \Phi_n : \Phi_i \in \lambda(a_i)\}$$

Определение 4.2.4: Язык, порождаемый грамматикой G

$L(G) = \{a_1 a_2 \dots a_n \in \Sigma^+ : \text{существуют } \Phi_i \in \lambda(a_i) \text{ такие, что } \Phi_1 \Phi_2 \dots \Phi_n \vdash^* S\}$

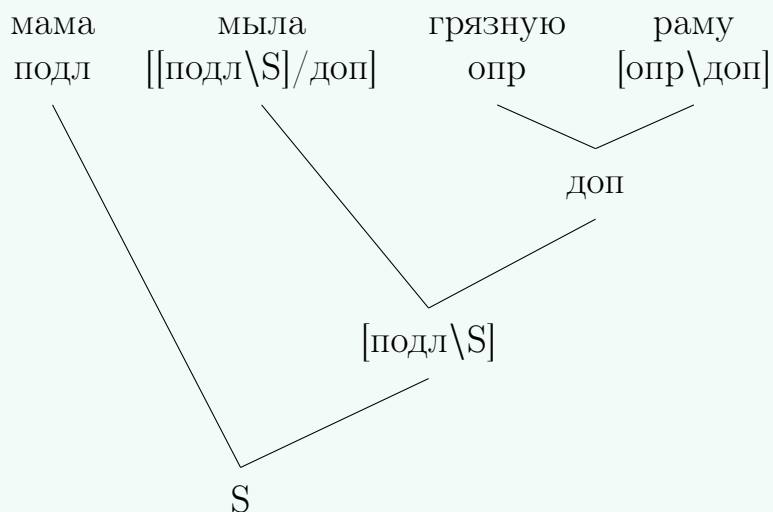
Пример 4.2.1

мама $\rightarrow$ подл

мыла $\rightarrow [\text{подл}\backslash S], [[\text{подл}\backslash S]/\text{доп}]$

грязную $\rightarrow$ опр

раму $\rightarrow$ доп, [опр\доп]



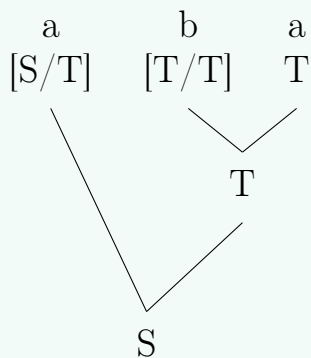
Пример 4.2.2

$L_1 = \{w \in \{a, b\}^+ : |w|_a \text{ чётно} \}$

$a \mapsto [S/T][T/S], T$

$b \mapsto [S/S], [T/T], S \text{ } aba$

$[S/T][T/T]T \vdash [S/T]T \vdash S$



Пример 4.2.3

$L_2 = \{a^n b^n : n > 0\}$

$a \mapsto [S/B]$

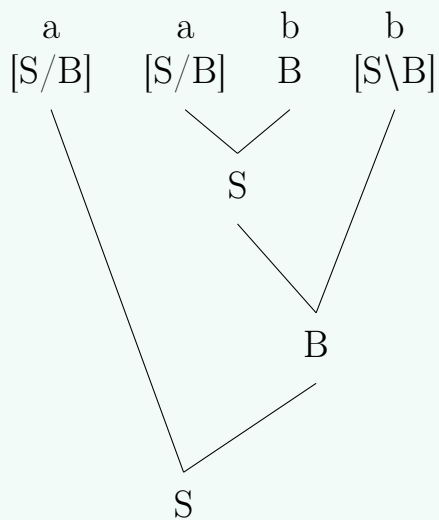
$b \mapsto [S\B], B$

ab

$[S/B]B \vdash S$

$aabb$

$[S/B][S/B]B[S\B]$



Вопрос 12: Построить категориальную грамматику для языка

$$L_3 = \{a^n c^k b^n : n, k > 0\}$$

$$a \mapsto [S/B]$$

$$b \mapsto [C\backslash B], [S\backslash B]$$

$$c \mapsto C, [C\backslash C]$$

[illegible]
$$L_4 = \{a^n b^k : 0 < n < k\}$$
$$b \mapsto [S \setminus B], [B'/B'], B', [B/B']$$

$L_5 \subseteq \{0, 1\}^+$ — язык правильной расстановки скобок.

$$10 \notin L_5$$
$$1 \mapsto A$$

$$2. [\Phi/\Psi], \Psi \in N \Rightarrow \Phi \rightarrow [\Phi/\Psi]\Psi \in P$$

$$3. [\Psi \setminus \Phi], \Psi \in N \Rightarrow \Psi[\Psi \setminus \Phi] \in P$$

Утверждение: Пусть $\Phi'_1, \dots, \Phi'_n, \Psi \in N$. Тогда $\Phi'_1, \dots, \Phi'_n \Psi \vdash^* \Psi \Leftrightarrow \Psi \Rightarrow^* \Phi_1 \dots \Phi_n$

Доказательство: Индукция по n

Базис: $n = 1, \Phi_1 = \Psi$

Индукционный шаг: $(\Rightarrow)\Phi_1 \dots \Phi_n \vdash^* \Psi$

$$\overline{a)\Phi_1 \dots \Phi_i \vdash^* [\Psi/\Theta]}$$

$$\Phi_{i+1} \dots \Phi_n \vdash^* \Theta$$

По индукционному предположению:

$$[\Psi/\Theta] \Rightarrow^* \Phi_1 \dots \Phi_i;$$

$$\Theta \Rightarrow^* \Phi_{i+1} \dots \Phi_n$$

$$\Theta \in N$$

$$\Psi \Rightarrow [\Psi/\Theta]\Theta \Rightarrow^* \Phi_1 \dots \Phi_n$$

$$б)\Phi_1 \dots \Phi_i \vdash^* \Theta$$

$$\Phi_{i+1} \dots \Phi_n \vdash^* [\Theta \setminus \Psi]$$

Аналогично

$$(\Leftarrow)\Psi \Rightarrow^* \Phi_1 \dots \Phi_n$$

$$a)\Psi \rightarrow [\Psi/\Theta]\Theta$$

$$[\Psi/\Theta] \Rightarrow^* \Phi_1 \dots \Phi_i$$

$$\Theta \Rightarrow^* \Phi_{i+1} \dots \Phi_n$$

По индукционному предположению:

$$\Phi_1 \dots \Phi_i \vdash^* [\Psi/\Theta]$$

$$\Phi_{i+1} \dots \Phi_n \vdash^* \Theta$$

$$\Phi_1 \dots \Phi_n \vdash^* [\Psi/\Theta]\Theta \vdash \Psi$$

$$б)\Psi \rightarrow \Theta[\Theta \setminus \Psi]$$

Аналогично.

